
CLIENT - SERVER

Esercizi basati su clientUDP e serverUDP

- 1) Lanciare prima il server e poi il client. Cosa si osserva? Invertire la sequenza di lancio. Cosa si osserva?
- 2) Modificare i sorgenti per mettere il server che riceve sulla porta 10000 e il client che trasmette dalla propria porta 30000 (ogni modifica dei sorgenti richiede una loro ricompilazione)
- 3) Mettere il server in ascolto sulla porta 100 e osservare cosa succede
 - Bisogna modificare anche il client? Dove?
 - Per chi usa il proprio PC con Linux o una virtual machine Linux, lanciare il server con il comando "sudo ./serverUDP" e osservare cosa cambia
- 4) Sostituire "127.0.0.1" prima con "localhost" e poi con "pippo" e osservare cosa succede
- 5) [Da fare solo se in Lab Delta] Accordarsi per lavorare su coppie di macchine in modo che server e client siano su macchine diverse. Come bisogna modificare i sorgenti?
- 6) Lanciare due volte il server usando due terminali. Cosa si osserva? Funzionano entrambi?
- 7) Modificare il server in maniera che soddisfi 5 richieste prima di terminare
 - E se volessi che non terminasse mai?

Esercizi su UDP

- 7) Compilare ed eseguire il secondo esempio
- 8) Modificarlo per costruire una semplice sommatrice
 - Il client acquisisce ripetutamente da tastiera un numero intero e lo manda al server finché l'utente digita zero
 - Il server accumula in una variabile "somma" i valori mandati dal client finché il client manda zero
 - Quando il client manda zero il server risponde al client con la somma ottenuta

Sommatrice UDP e perdita di pacchetti

- 9) Usare la sommatrice su due macchine distinte provando, sulla macchina del client, a staccare il cavo di rete prima di un invio di un dato, ad es.
 - Digitare "2345" + INVIO
 - Digitare "5187" + INVIO
 - Staccare il cavo
 - Digitare "2" + INVIO
 - Riattaccare il cavo e aspettare 30 sec che il sistema operativo si riassesti
 - Digitare "1" + INVIO
 - "0"
 - Che somma leggo? E' corretta?
 - Se il server gira su una macchina Linux (anche una macchina virtuale) e comunica con il client attraverso una rete WiFi

oppure in loopback è possibile simulare la perdita di pacchetti bloccando e, dopo poco tempo, sbloccando la ricezione di pacchetti con i seguenti comandi:

- PER BLOCCARE: sudo iptables -A INPUT -p udp --dport <porta_del_server> -j DROP
- PER SBLOCCARE: sudo iptables -D INPUT -p udp --dport <porta_del_server> -j DROP
- Si consiglia di prepararsi questi comandi in una shell per poterli dare velocemente

sommatrice UDP e influenze reciproche

10)

- Invocare il server della sommatrice con due client diversi (tutti e tre possono anche essere sulla stessa macchina ovviamente su finestre terminali diverse), ad es.
- Che somma leggo da ciascun client? E' la somma che ciascun client da solo si aspetterebbe?

Client A:

- Digitare "2345" + INVIO
- Digitare "5187" + INVIO
- Digitare "2" + INVIO
- Digitare "1" + INVIO
- "0"

Client B:

- Digitare "2" + INVIO
- Digitare "8" + INVIO
- "0"

Esercizi su TCP

11) Lanciare due volte il server usando due terminali. Cosa si osserva? Funzionano entrambi?

12) Scrivere la sommatrice usando TCP, compilare ed eseguire

13) Provare a rifare il caso dell'Esercizio 10 ma con questa nuova versione della sommatrice.

Cosa si può osservare? Che soluzione si può trovare? C'è influenza reciproca tra i due client?

14) Usare la sommatrice TCP su due macchine distinte provando, sulla macchina del client, a staccare il cavo di rete prima di un invio di un dato, ad es.

- Digitare "2345" + INVIO
- Digitare "5187" + INVIO
- Staccare il cavo
- Digitare "2" + INVIO
- Riattaccare il cavo e aspettare 30 sec che il sistema operativo si riassesti
- Digitare "1" + INVIO
- "0"

Che somma leggo? E' corretta?

- NOTA: Se il server gira su una macchina Linux (anche una macchina virtuale) e comunica con il client attraverso una rete WiFi

oppure in loopback è possibile simulare la perdita di pacchetti bloccando e, dopo poco tempo, sbloccando la ricezione di pacchetti con i seguenti comandi:

- PER BLOCCARE: sudo iptables -A INPUT -p tcp --dport <porta_del_server> -j DROP
- PER SBLOCCARE: sudo iptables -D INPUT -p tcp --dport <porta_del_server> -j DROP

- Si consiglia di prepararsi questi comandi in una shell a parte per poterli dare velocemente

applicazione di trasferimento file

- Il client chiede al server un file specificandone il nome
 - Il server lo trasmette un byte alla volta
 - Il client lo salva in locale con lo stesso nome
 - Quale protocollo usiamo?
 - [Da fare solo con i PC del Lab Delta] Lanciare client e server su due macchine diverse, trasferire un file di grosse dimensioni in modo da avere il tempo di staccare e riattaccare il cavo di rete per un breve tempo durante il trasferimento.
- Cosa succede al file trasferito?

Temi d'esame

(per ognuno ragionare sulla scelta del protocollo di livello trasporto)

- Implementare un servizio di calcolo remoto: il client spedisce al server in un unico messaggio due numeri interi e il tipo di operatore (somma, sottrazione, moltiplicazione, divisione) e il server, dopo aver svolto il calcolo, restituisce il risultato al client che così termina la sua esecuzione.
- Implementare un servizio di calcolo remoto: il client spedisce al server in 3 distinti messaggi due numeri interi e il tipo di operatore (somma, sottrazione, moltiplicazione, divisione) e il server, dopo aver svolto il calcolo, restituisce il risultato al client che così termina la sua esecuzione.
- Implementare un servizio di telepass: quando l'auto si avvicina al casello il terminale in esso presente spedisce al server il numero di targa; il server tiene traccia in una struttura dati del numero di passaggi per ogni targa e restituisce al client tale numero.
- Implementare una semplice versione di terminale remoto. Il client si connette ad un server di cui si conosce IP e porta e chiede all'utente di scrivere un comando come in un normale terminale a caratteri (detto anche shell o console) di Linux. Il comando viene eseguito dal server e il relativo output viene stampato a video dal client.

WEB

Esercizio

- Scrivere e provare una pagina HTML che ricarica periodicamente il sito dell'ANSA

Un semplice web server

- Aprire il file serverHTTP.c in Esempi-web/ e analizzarne il contenuto
- Compilarlo come nell'esercitazione sull'interfaccia socket ed eseguirlo
- Provare ad aprire un secondo terminale nella stessa cartella e a rilanciare lo stesso server. Funziona? Perché?

- Aprire il browser preferito e impostare la URL "http://127.0.0.1:8000/" Cosa si vede sul browser e sul terminale?
- Aprire il browser preferito e impostare la URL "http://localhost:8000/" Cosa cambia?

```
form-get.html
<!DOCTYPE html><html>
<body>
<h2>The method Attribute</h2>
<p>This form will be submitted using the GET method:</p>
<form action="/action" target="_blank" method="get">
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" value="John"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname" value="Doe"><br><br>
<input type="submit" value="Submit">
</form>
<p>After you submit, notice that the form values is visible in the address bar
of the new browser tab.</p>
</body>
</html>
```

Esercizio

- Eseguire il server web serverHTTP.c e con il browser preferito aprire il file form-get.html
- Cosa si vede?
- Provare ad analizzare il contenuto della connessione TCP con Wireshark.
- Cosa si vede?

```
form-post.html
<!DOCTYPE html><html>
<body>
<h2>The method Attribute</h2>
<p>This form will be submitted using the GET method:</p>
<form action="/action" target="_blank" method="post">
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" value="John"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname" value="Doe"><br><br>
<input type="submit" value="Submit">
</form>
<p>After you submit, notice that the form values is visible in the address bar
of the new browser tab.</p>
</body>
</html>
```

Esercizio

- Eseguire il server web serverHTTP.c e con il browser preferito aprire il file form-post.html
- Cosa si vede?
- Provare ad analizzare il contenuto della connessione TCP con Wireshark.
- Cosa si vede?

Esercizio

- Modificare il file `serverHTTP.c` in modo che, invece di restituire sempre la solita pagina web di prova, restituisca una delle pagine html usate dal lucido 9 in poi scelta attraverso l'uso del browser.
- Suggestimenti:
 - Provare a fare la nuova richiesta col browser usando il `serverHTTP.c` in modo da vedere la richiesta HTTP per capire dove si trova la stringa con il nome del file nella richiesta che fa il browser (aiutarsi anche con Wireshark)
- Cosa devo scrivere nella barra del browser?
 - Capire come recuperare la stringa con il nome del file dalla richiesta HTTP (si veda `esempio-parser.c`)
 - Per costruire la risposta riciclare parte del codice usato nell'esercizio del trasferimento di file
- Prova finale: cosa succede se chiedo al server di restituire i file `form-get.html` e `form-post.html` ?

Esercizio per casa

- Modificare il server web `serverHTTP.c` in modo che accetti con il metodo POST un intero file da salvare nella cartella del server (il cosiddetto "upload sul server").
- Suggestimenti:
 - utilizzare il file `form-file.html` con `serverHTTP.c` non modificato ed analizzare il contenuto della connessione TCP con Wireshark in modo da capire nella richiesta HTTP
- Dove si trova il nome del file
- Dove si trova il contenuto del file
 - Nella lettura della richiesta HTTP sul server aggiungere il codice che salva il file prendendo spunto dal codice usato nell'esercizio del trasferimento di file
 - Invece la risposta HTTP può essere molto statica come nella versione originale di `serverHTTP.c`

Esempio di web server esteso con gestione CGI

- Aprire il file `serverHTTP-CGI.c` in `Esempi-web/` e analizzarne il contenuto
- Compilarlo come al solito ed eseguirlo
- Aprire il browser preferito e il file `sommatrice-web.html`
 - Cosa si vede sul browser?
 - NOTA: Provare con numeri positivi, negativi, con parte decimale...
- A cosa corrisponde il secondo parametro della funzione `sommatrice()` ?

Esercizio su Web Socket

- Realizzazione di una chat eseguita dentro un browser web
 - Fare riferimento al materiale apposito condiviso con gli studenti

Esercizio

- Considerare la cartella `WebService/`
- Aprire il file `serverHTTP-REST.c` e analizzarne il contenuto
- Compilarlo come al solito ed eseguirlo
- Aprire il file `clientREST-GET.c` e analizzarne il contenuto
- Compilarlo come al solito ed eseguirlo
 - Che parametri devo passare in linea di comando?
 - Cosa si può vedere analizzando lo scambio di dati tramite Wireshark?
 - Qual è la signature della funzione `calcolaSomma()` sul server e sul client? Perché ha senso che siano uguali?

Esercizio 2

- Prendere in considerazione il file `ClientREST.java`

- Dopo aver installato l'ambiente base di Java (già presente in Lab Delta) si può compilare con `javac ClientREST.java`
- Ed eseguire con `java ClientREST`
- Il fatto che il server sia fatto in C e il client in Java è un problema o un'opportunità? Perché?

Esercizio 3

- Estendere il webservice `serverHTTP-REST.c` in modo che esponga un secondo servizio relativo al calcolo dei numeri primi compresi nell'intervallo `[min, max]`
- Si tragga spunto al programma `prime-number-interval.c`
- Estendere il file `ClientREST.java` in modo da poter chiamare, a scelta, entrambe le funzionalità della nuova API
- Provare il client con il calcolo dei numeri primi compresi nell'intervallo `[1, 1000000]`
- Quanto tempo ci mette?
- E' stato necessario tradurre l'algoritmo dei numeri primi in Java? Perché?

Esercizio finale

- Prendere in considerazione il file `clientThreadREST.java`
- Esso invoca `calcolaSomma()` in 3 thread concorrenti
- Però la macchina su cui gira il server chiamato è la stessa e quindi non ci guadagno in prestazioni
- Come si potrebbe modificare il codice in modo che le 3 invocazioni finiscano su 3 macchine diverse?
- Bisogna modificare anche il codice del server?
- Si riconsideri consideri il servizio che calcola i numeri primi nell'intervallo `[min, max]` costruito nell'esercizio precedente
- Si trovi un modo efficiente, sfruttando diversi server in rete, per calcolare il numeri primi tra 1 e 1000000
- Di quanto migliorano le prestazioni?
- Devo modificare anche il codice del server?

WEBSOCKET

Esercizio 0

Effettuare una cattura Wireshark relativa al funzionamento della chat. Occorre attivare la cattura prima di aprire la URL dalla finestra del browser e ascoltare non solo sull'interfaccia di loopback ma anche su quella verso l'esterno (opzione ANY).

Esercizio 1

Modificare a piacimento il contenuto del file `public/index.html` e valutarne l'impatto grafico.

Esercizio 2

Modificare il sorgente del codice in modo da far ascoltare il server sulla porta 80 invece che 4000. Quali altri accorgimenti sono necessari per farlo funzionare: lato server? Lato client?

Esercizio 3

Modificare il sorgente del codice in modo da cambiare nome ai seguenti eventi
`message` → messaggio
`UploadChat` → aggiornamento

Esercizio 4

Se si dispone di due PC in grado di dialogare sulla stessa rete IP (oppure un PC

per il server e uno smartphone per il client sempre in grado di dialogare tra loro a livello IP) provare ad accedere al server che è su Linux mediante diversi tipi di browser e di sistemi operativi. Cosa si può notare?

Esercizio 5

Modificare il sorgente del codice per fare in modo che ad ogni utente connesso alla chat arrivi nella console il messaggio "l'utente sta scrivendo..." .

NOTA: Lato client, bisogna spedire al server un evento apposito (ad es. "typing") quando l'utente scrive sulla tastiera (catturando l'evento di sistema "keydown"). Lato server, la chiamata `websocket.broadcast.emit('typing', data)` rilancia l'evento "typing" a tutti i client connessi tranne che a quello dalla quale si è ricevuto il messaggio.

Lato client occorre infine gestire la ricezione del messaggio "typing" che arriva dal server (vedere gestione del messaggio "UploadChat").

Esercizio 6 (manca l'esercizio chat in tcp nel lab client-server)

Pensando all'esercizio sulla chat dell'esercitazione sulla programmazione mediante socket, quali differenze/vantaggi/svantaggi si hanno con le tecnologie impiegate in questa esercitazione?

MQTT

- Partendo dall'esercizio mostrato nelle istruzioni, cosa succede se pubblico una temperatura prima di aver lanciato il subscriber?
 - Provare il publisher con l'opzione `--retain`
- Partendo dall'esercizio mostrato nelle istruzioni creare un'applicazione pub/sub con 2 sensori di temperatura relativi a 2 stanze diverse. Quante finestre di terminale devo aprire?
- Partendo dall'esercizio precedente come fare per avere un unico subscriber per entrambe le temperature? Come si fa a distinguere da quale stanza proviene la temperatura?
- Prova a pubblicare un valore di umidità relativa (topic "UR"); il subscriber interessato alle temperature lo riceve? Come si fa a creare un subscriber interessato all'umidità? Costruire un'applicazione pub/sub con 4 finestre per produrre e visualizzare sia valori di temperatura sia valori di umidità.
- Aprire e studiare con Wireshark il file PCAP contenuto nello ZIP
 - Quale protocollo di livello trasporto utilizza MQTT? Quali sono le porte?
 - Trovare le fasi di publish e subscribe. Quante connessioni TCP apre un subscriber? Quante connessioni TCP apre un publisher?

Si vuole costruire con MQTT un servizio di messaggistica universitaria

- Il rettore può leggere tutti i messaggi
- La segreteria può leggere i messaggi dai docenti e dagli studenti
- I docenti possono leggere i messaggi dai docenti e dagli studenti
- Gli studenti possono leggere solo i messaggi degli altri studenti